

Neural networks

Hakyung Sung
University of Hawaii at Manoa

September 30, 2026

Topics to cover

- Review
- Neural network

Review: Word vectors

Word vectors

- Distributional semantics
- Vector semantics

Distributional semantics

*...government debt problems turning into **banking** crises as happened in 2009...*
*...saying that Europe needs unified **banking** regulation to replace the hodgepodge...*
*...India has just given its **banking** system a shot in the arm...*

These **context words** will represent **banking**

“You shall know a word by the company it keeps” (Firth, 1957)

Related terms

- Words cannot be given as inputs to computers, but *numbers* can.
- Encoding: converting words to numbers
- Vector: an ordered list of numbers (e.g., [0.1, 0.3, -0.5])
- One-hot encoding: a simple encoding (for word positions; **no contextual information**)

Neural network-based models

Neural models learn vectors directly by predicting context words

- 1986: Learning representations by back propagating errors (Rumelhart et al., 1986)
- 2003: A neural probabilistic language model (Bengio et al., 2003)
- 2013: **Word2Vec** (Skip-gram, CBOW)
- 2014–2015: GloVe, fastText
- 2018– : Contextual embeddings (ELMo, BERT, GPT)

Word2Vec: Skip-grams (*window size* = 2)

- “king brave man”
- “queen beautiful woman”

word	neighbor
king	brave
king	man
brave	man
brave	king
man	king
man	brave
queen	beautiful
queen	woman
beautiful	queen
beautiful	woman
woman	queen
woman	beautiful

Word2Vec: Skip-grams (window size = 2)

word	one-hot encoding	neighbor	one-hot encoding
king	[1, 0, 0, 0, 0, 0]	brave	[0, 1, 0, 0, 0, 0]
king	[1, 0, 0, 0, 0, 0]	man	[0, 0, 1, 0, 0, 0]
brave	[0, 1, 0, 0, 0, 0]	man	[0, 0, 1, 0, 0, 0]
brave	[0, 1, 0, 0, 0, 0]	king	[1, 0, 0, 0, 0, 0]
man	[0, 0, 1, 0, 0, 0]	king	[1, 0, 0, 0, 0, 0]
man	[0, 0, 1, 0, 0, 0]	brave	[0, 1, 0, 0, 0, 0]
queen	[0, 0, 0, 1, 0, 0]	beautiful	[0, 0, 0, 0, 1, 0]
queen	[0, 0, 0, 1, 0, 0]	woman	[0, 0, 0, 0, 0, 1]
beautiful	[0, 0, 0, 0, 1, 0]	queen	[0, 0, 0, 1, 0, 0]
beautiful	[0, 0, 0, 0, 1, 0]	woman	[0, 0, 0, 0, 0, 1]
woman	[0, 0, 0, 0, 0, 1]	queen	[0, 0, 0, 1, 0, 0]
woman	[0, 0, 0, 0, 0, 1]	beautiful	[0, 0, 0, 0, 1, 0]

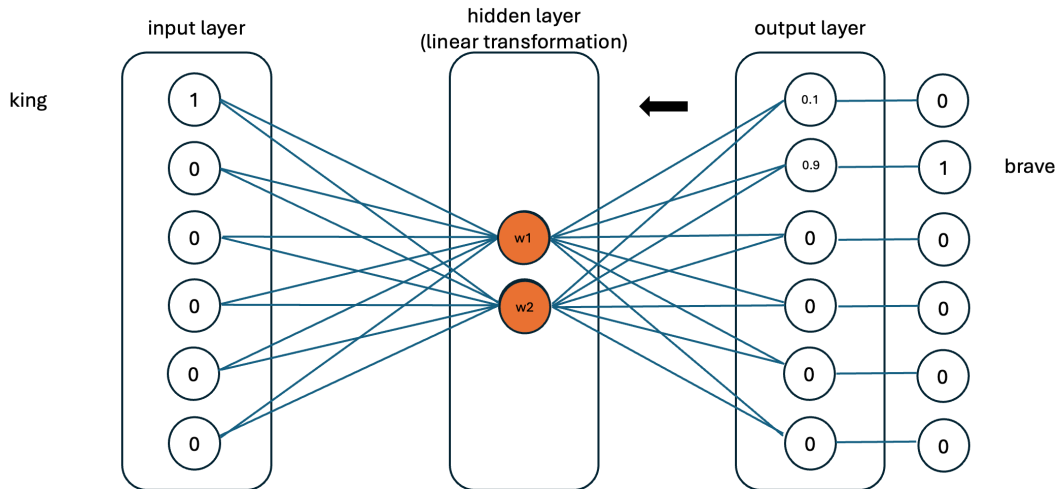
Word2Vec: Input and output

input
[1, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1]

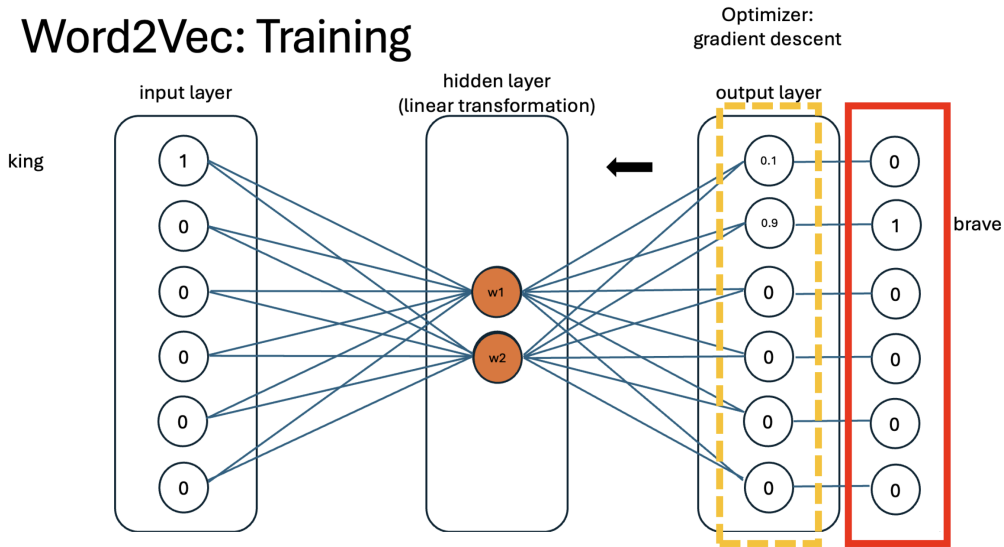
output
[0, 1, 0, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[0, 0, 1, 0, 0, 0]
[1, 0, 0, 0, 0, 0]
[1, 0, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 1]
[0, 0, 0, 0, 1, 0]

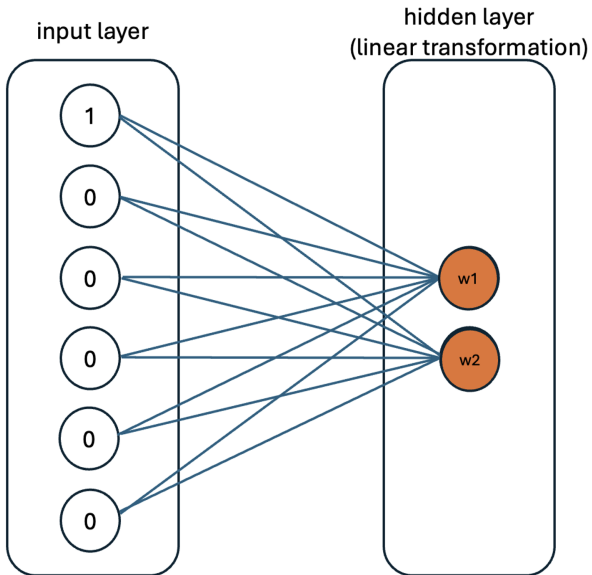
Word2Vec: Training

Optimizer:
gradient descent



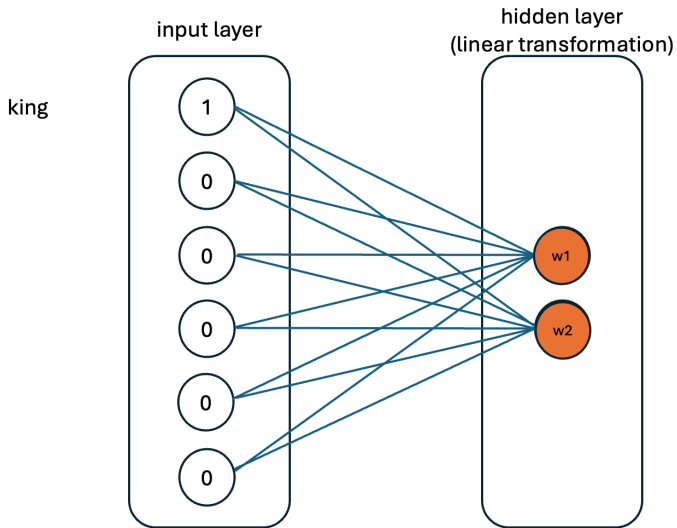
Word2Vec: Training



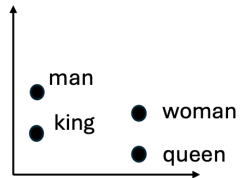


word	embedding
king	[1, 0]
brave	[2, 0]
man	[3, 0]
queen	[0, 0]
beautiful	[0, 0]
woman	[0, 0]





word	embedding
king	[1, 1]
brave	[1, 2]
man	[1, 3]
queen	[5, 1]
beautiful	[5, 2]
woman	[5, 3]

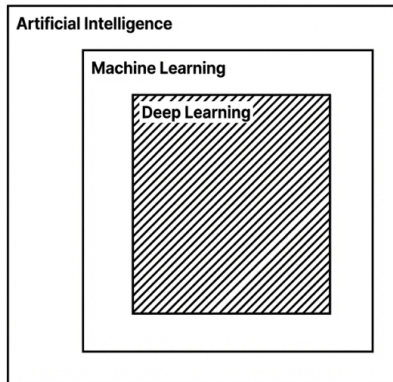


- The shift to neural representation learning marked a major turning point in NLP.
- **So, what is a neural network and how it works?**

Artificial neural network

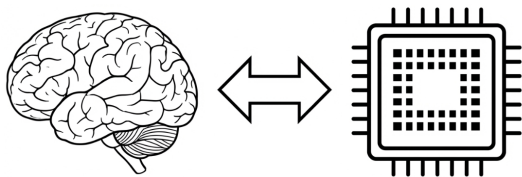
Terms

- Artificial intelligence (AI): The broad goal of creating thinking machines.
- Machine learning: Systems that learn from data.
- Deep learning: The subset built on *artificial neural network*.



Understanding human brain

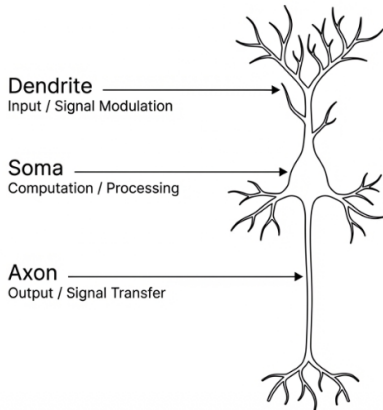
Artificial neural networks are designed to mimic the structure and function of biological processors (i.e., brain).



- Therefore, understanding how human brains work is the very first step.
- Human brains are quite complex.
- But, its basic unit is relatively simple.

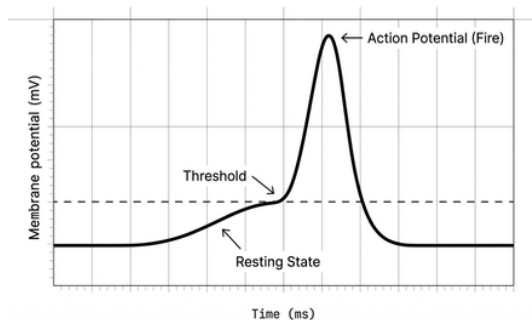
The biological processor: Neuron

A single neuron acts as an information-processing unit with three functions:
(1) input, (2) computation, (3) output



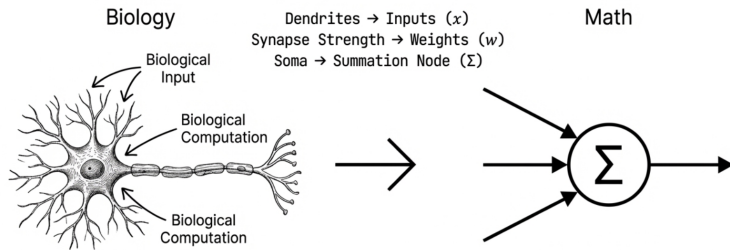
Firing

- Neurons do not simply pass signals along; they make decisions.
- In the soma, incoming signals are computed.
 - If this total signal exceeds a specific threshold, the neuron fires an “action potential”
 - if not, nothing happens.



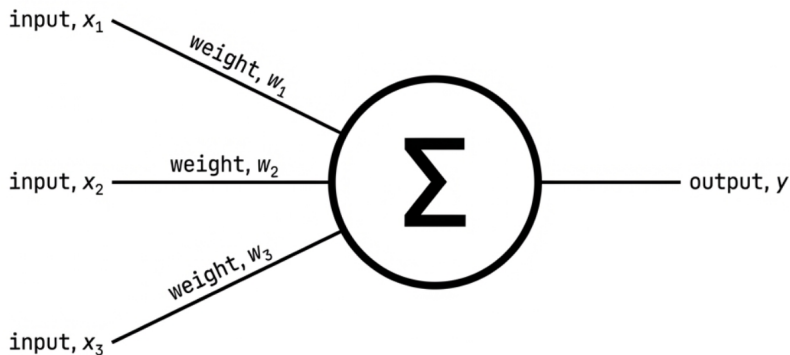
From biology to code

- Artificial neurons are designed to mimic the information-processing mechanisms of these neurons.
- We abstract the biology into a mathematical model called *perceptron*.



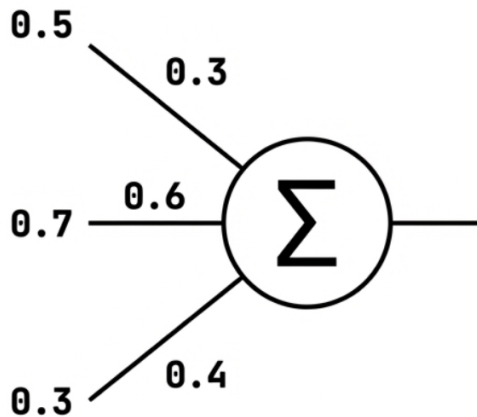
Perceptron

- The artificial node receives multiple inputs (x).
- Each input is multiplied by a corresponding weight (w).
- The circular node (Σ) sums these values.
- The result is passed to an activation function to determine the final output (y).



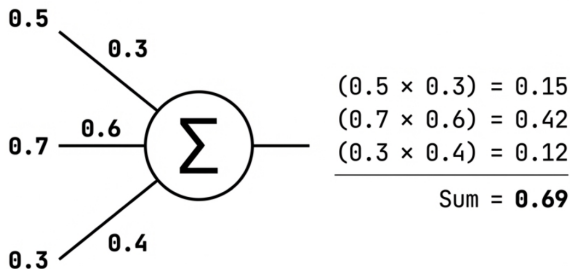
Perceptron: Example

Let's calculate the value for a single node using specific inputs and weights.

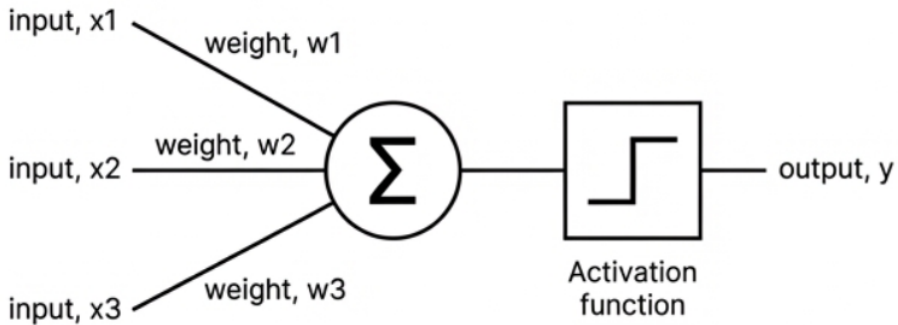


Perceptron: Example

- Let's calculate the value for a single node using specific inputs and weights.



- Weighted sum = 0.69 $\Rightarrow$ The neuron decides whether to fire.
- Apply a simple *step function* with a threshold of 0.5.



```
IF (Sum >= Threshold):
```

```
    Output = 1
```

```
ELSE:
```

```
    Output = 0
```

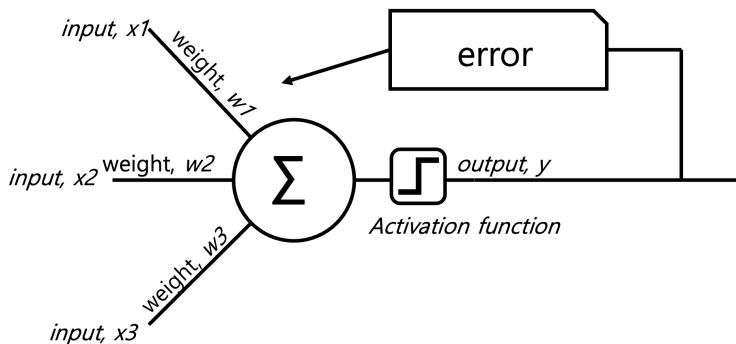
Calculated Sum: 0.69

Threshold: 0.5

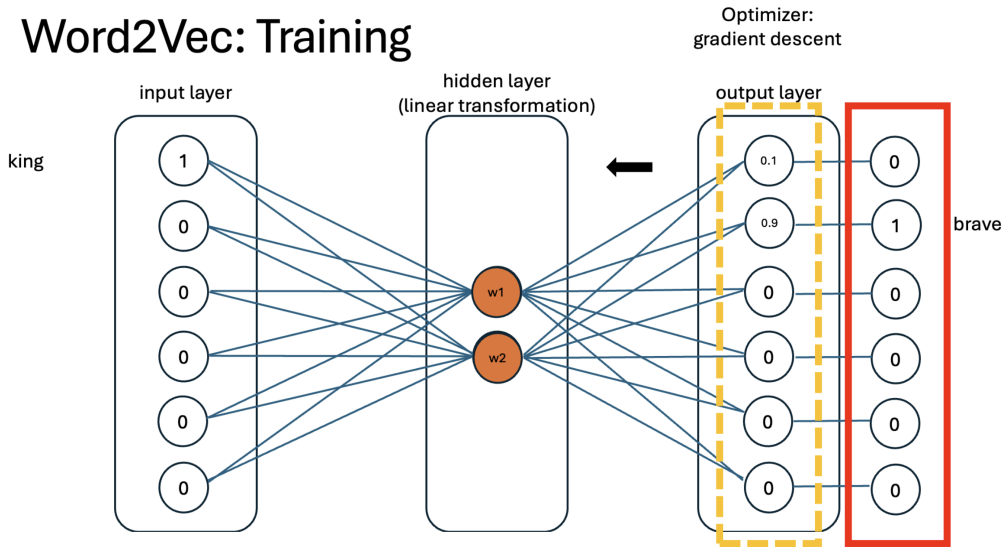
Result: 1 (The Neuron Fires)

Training perceptron

To train perceptron, we compare the predicted output with the actual output. This difference is the *error*, which is then used to **adjust the weights** so that the model improves over time.



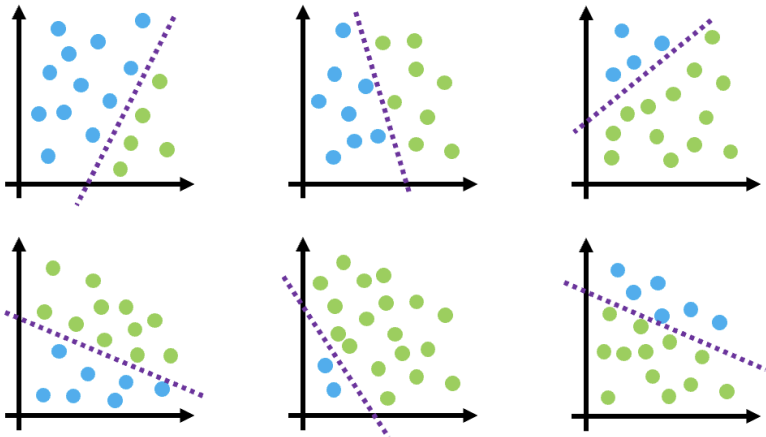
Word2Vec: Training



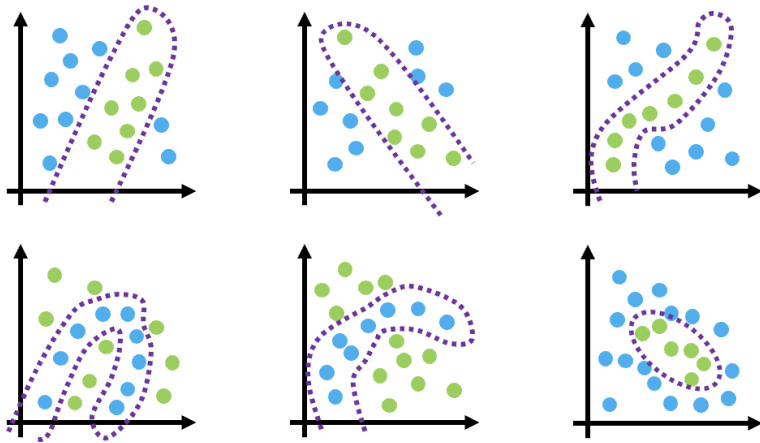
Single-layer perceptron

So far, we explored a single-layer perceptron.

- works well for linearly separable data.



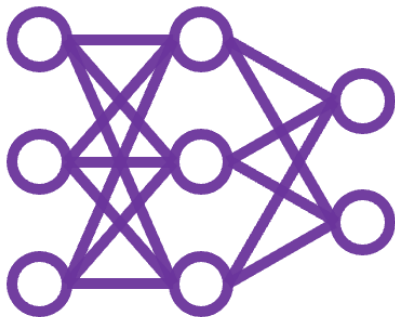
Single-layer perceptron: Limitation



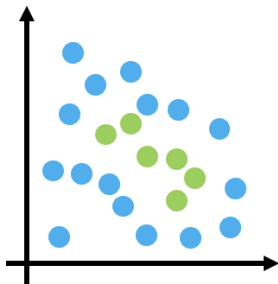
Multi-layer perceptron

Multi-layer perceptron (MLP)

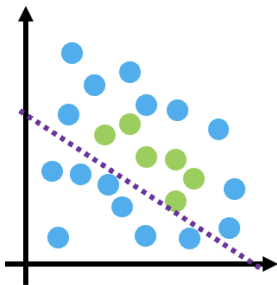
If we allow multiple lines, there is a possibility to separate even non-linear data.
This idea leads us to the **multi-layer perceptron (MLP)**.



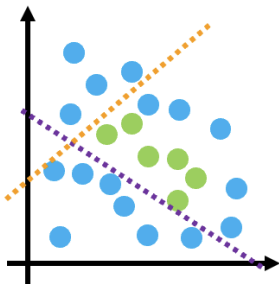
Let's assume we are given data in a complex form like this.



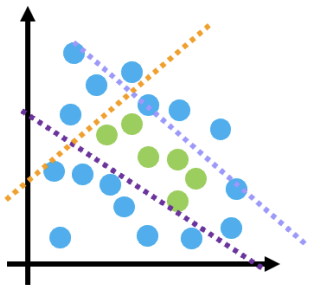
With a single perceptron, linear separation is not possible.



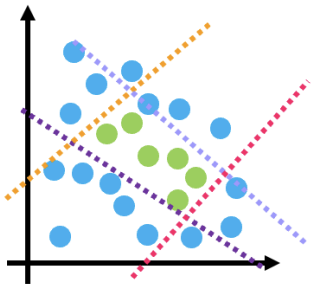
But if we add more lines, it becomes possible to separate further.



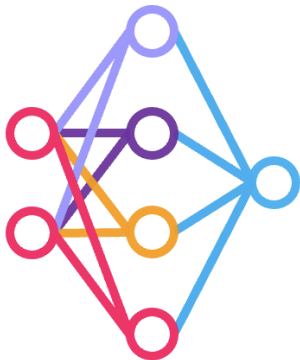
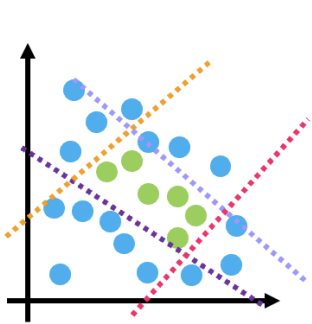
By adding several lines, the separation becomes more feasible.



Four lines can be thought of as the outputs of four perceptrons.

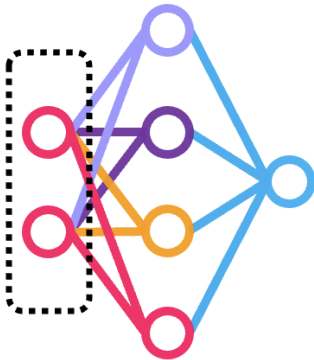


If we then connect another perceptron that takes these four outputs as its inputs, we can construct a **multi-layer neural network** capable of non-linear separation.



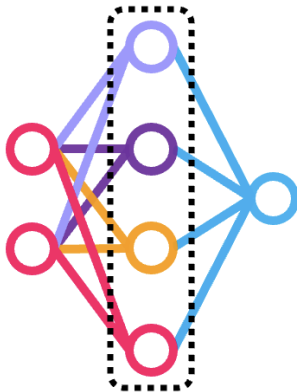
Structure of an MLP

So the MLP we build here consists of an input layer



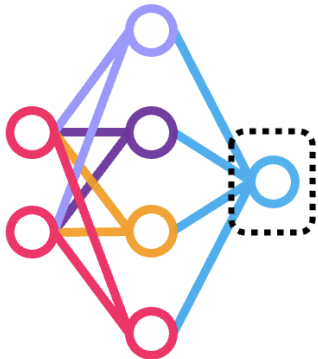
Structure of an MLP

a hidden layer



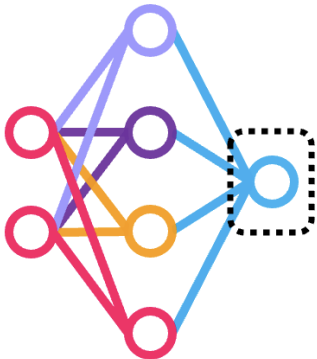
Structure of an MLP

and an output layer



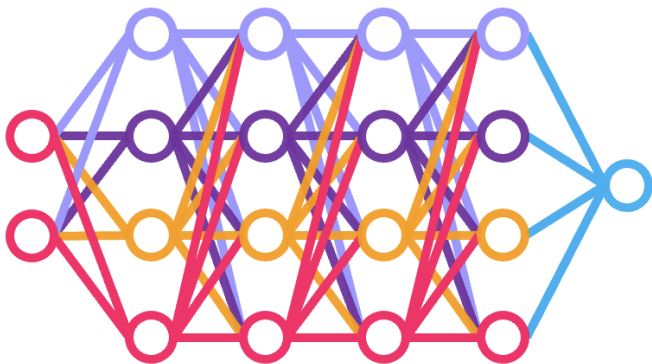
Structure of an MLP

As the number of layers increases, the model can handle more complex data.



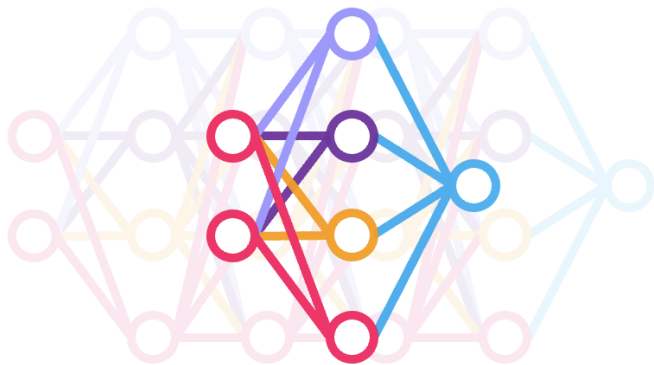
Structure of an MLP

When a network has many layers, we call it “deep.” This is where the term deep learning comes from.



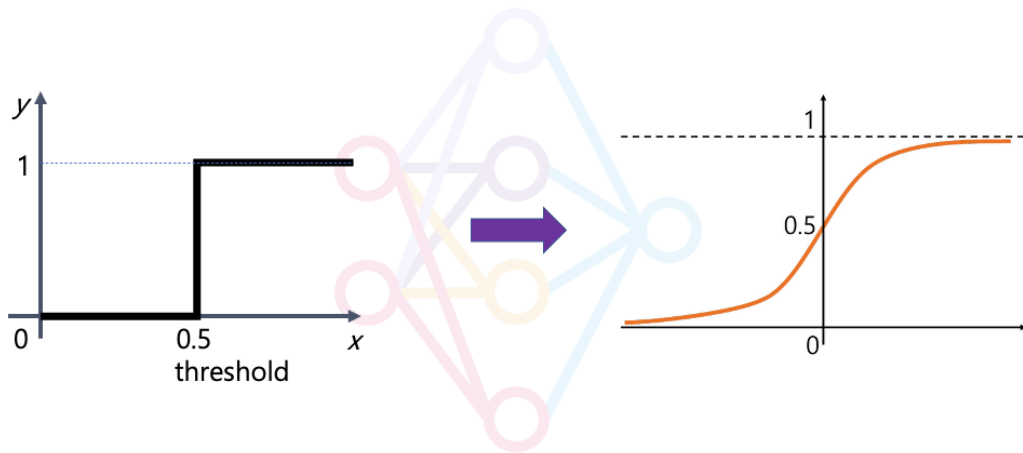
Structure of an MLP – Deep Learning

To understand how multilayer networks work, we need to look at a few more changes.



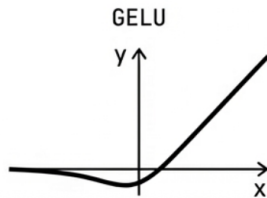
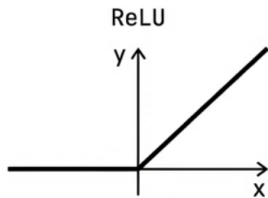
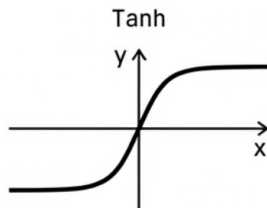
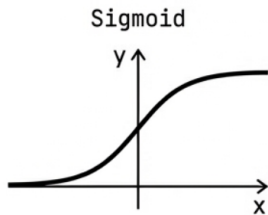
More changes: Activation function

More complex activation functions can be used.
e.g., *sigmoid function*



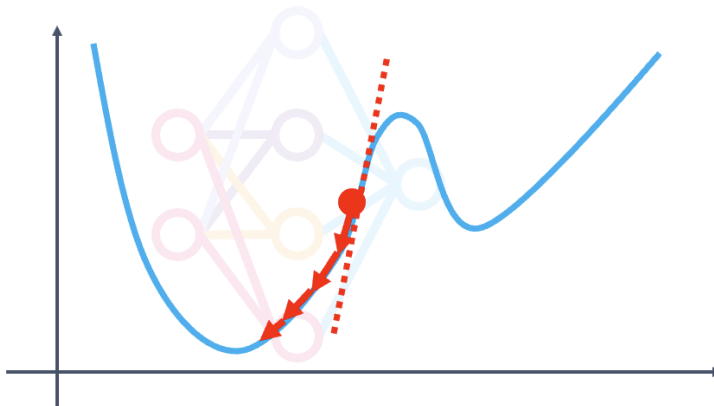
More: At-a-Glance Comparison

In fact, many more activation functions have been introduced:



More changes: Optimization

To reduce errors in multilayer networks, methods like gradient descent are used.



More changes: Optimization

- **Goal:** Learn good word vectors by minimizing a loss function $J(\theta)$ (measures how wrong predictions are).
- **Idea:**
 - Start from random initial values
 - Compute the gradient of $J(\theta)$ (which tells us the slope)
 - Move a small step in the *opposite direction* of the gradient
 - Repeat many times until the loss becomes small
 - **Gradient descent** is an optimization algorithm used to minimize the loss function.

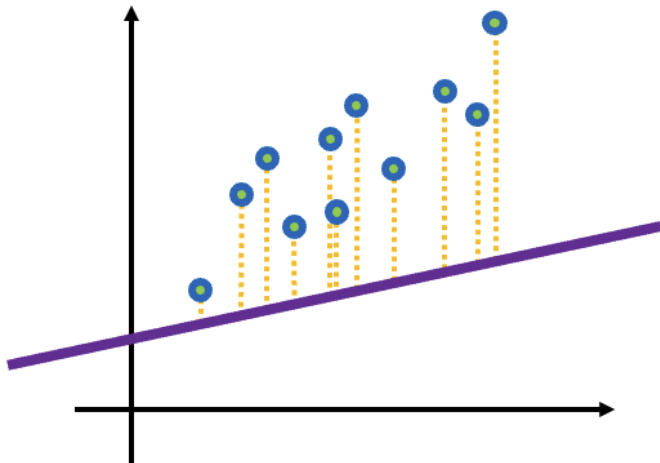
Gradient descendant and loss function

Gradient Descent: Definition

Gradient descent optimization minimizes a given **loss function** by updating model parameters.

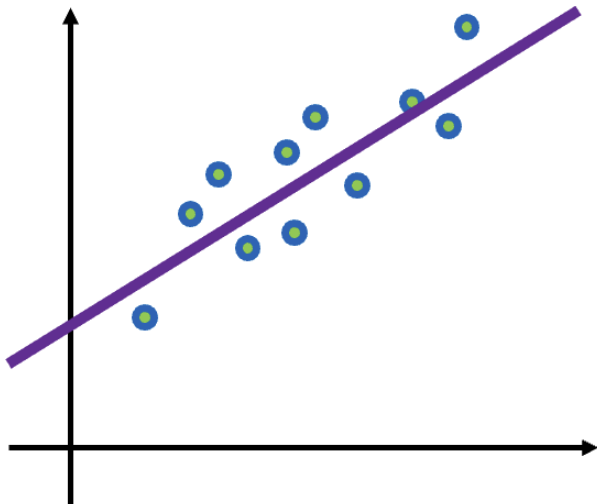
Loss Function

A loss function measures the difference between predicted values ($\hat{y}$) and actual values (y).



Loss Function

After training, if the relationship (between predicted values ($\hat{y}$) and actual values (y)) changes like this, the error will likely decrease.

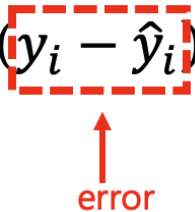


Loss Function

- Example: **Mean Squared Error (MSE)**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Loss Function: MSE

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$


error

Loss Function: MSE

To remove the effect of the sign, square the difference.

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

Loss Function: MSE

Add up the errors for all the data points.

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

Loss Function: MSE

Divide by the number of data points.

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

Loss Function

- Other common loss functions: *Cross-Entropy*
- **Smaller** loss values indicate better model performance!

Gradient Descent: Core Idea

Now that we understand the loss function, let's think about how to apply the gradient descent algorithm using MSE.

$$MSE = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

Gradient Descent: Core Idea

To make things simple, let's first consider the case where we only have one data point.

$$MSE = \frac{1}{1} \sum_1^1 (y_1 - \hat{y}_1)^2$$

Gradient Descent: Core Idea

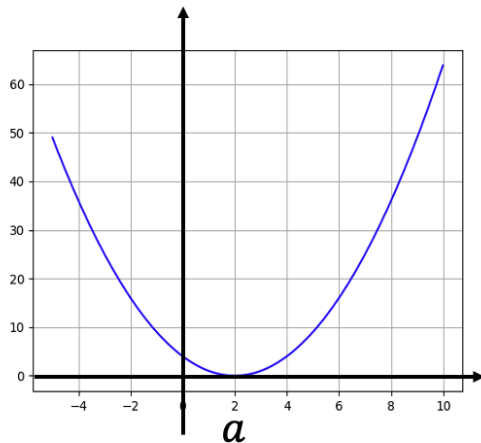
In this case, it reduces to a familiar quadratic equation.

$$MSE = \frac{1}{1} \sum_1^1 (y_1 - \hat{y}_1)^2$$
$$y = \frac{1}{1} \sum_1^1 (x_{\square} - a)^2$$

Gradient Descent: Core Idea

If we plot this as a graph, it looks like this.

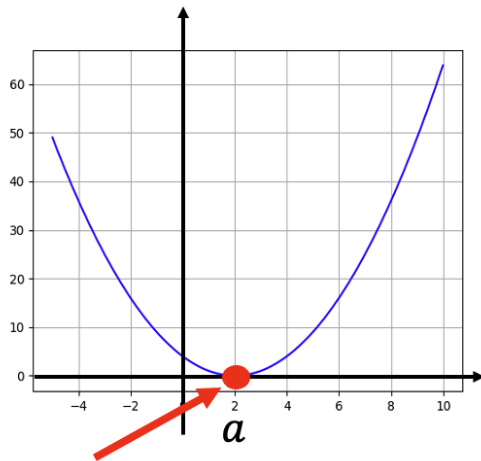
$$MSE = \frac{1}{1} \sum_1^1 (y_1 - \hat{y}_1)^2$$
$$y = \frac{1}{1} \sum_1^1 (x_{\square} - a)^2$$



Gradient Descent: Core Idea

The point where the error is minimized is here.

$$MSE = \frac{1}{1} \sum_1^1 (y_1 - \hat{y}_1)^2$$
$$y = \frac{1}{1} \sum_1^1 (x_{\square} - a)^2$$

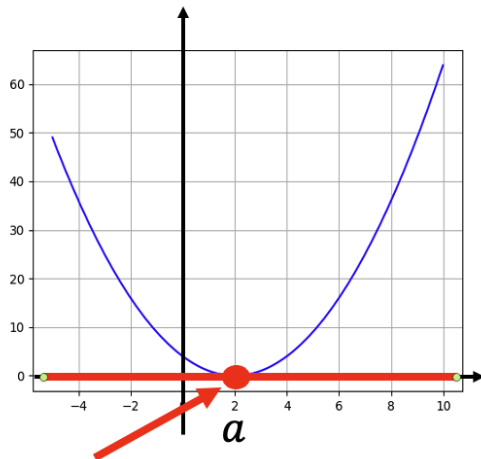


Gradient Descent: Core Idea

At the minimum error, the slope of the curve is zero.

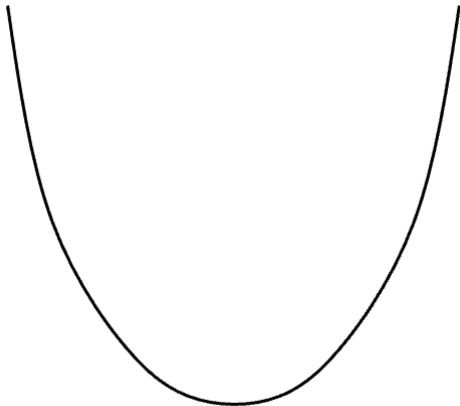
$$MSE = \frac{1}{1} \sum_1^1 (y_1 - \hat{y}_1)^2$$

$$y = \frac{1}{1} \sum_1^1 (x_{\square} - a)^2$$



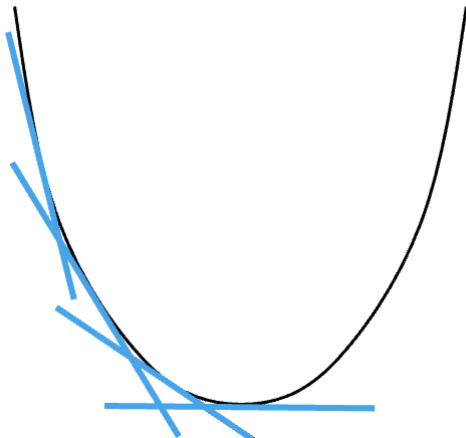
Gradient Descent: Core Idea

So the goal is to find the point where the tangent slope becomes zero.



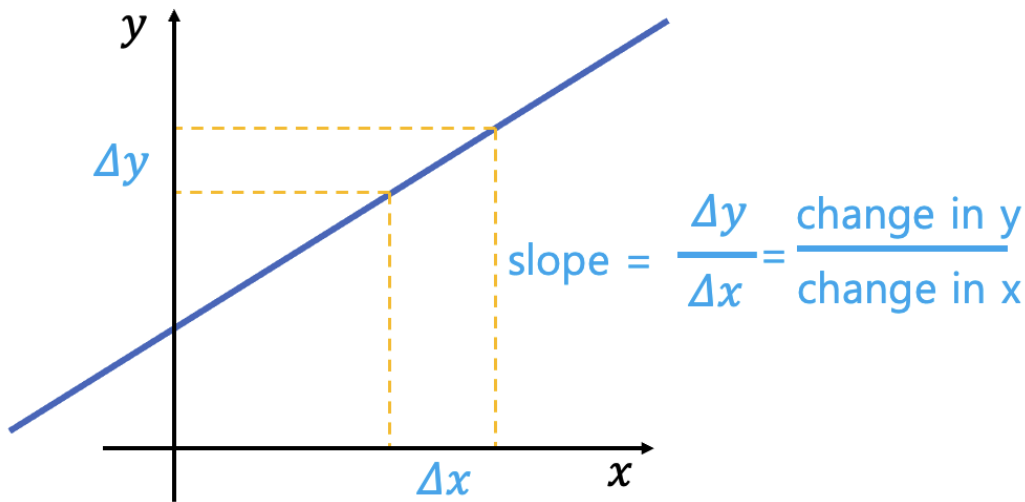
Gradient Descent: Core Idea

The method of gradually decreasing the slope of the tangent is what we call gradient descent.



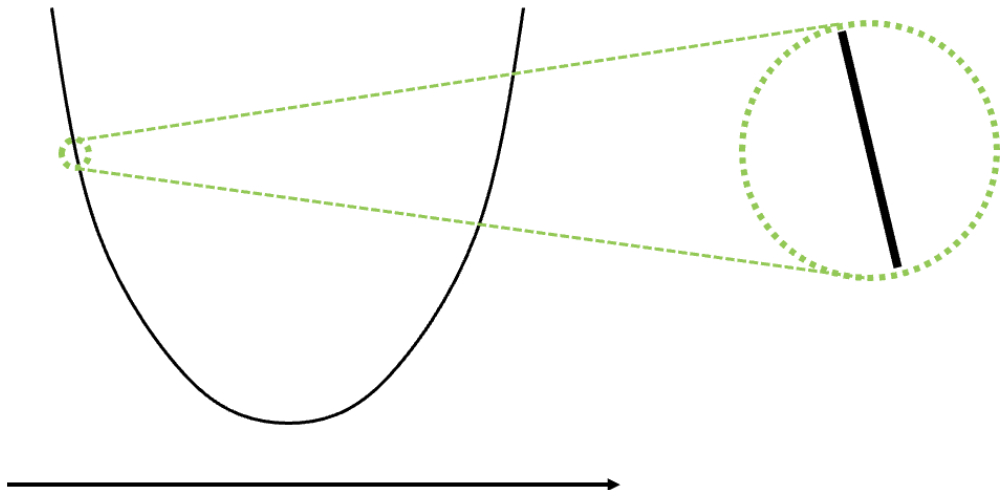
Gradient Descent: Core Idea

For a linear function, we can express the slope like this.



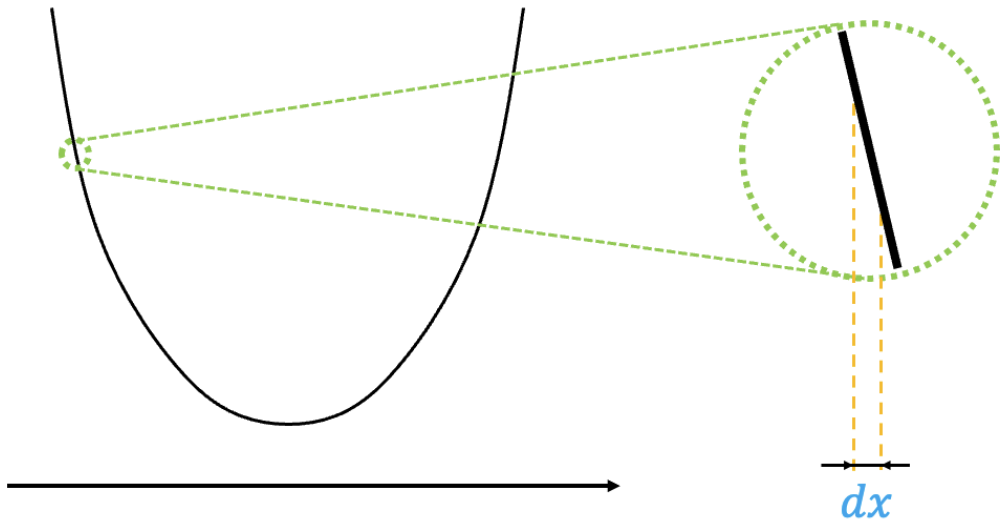
Gradient Descent: Core Idea

And by using derivatives, we can compute slopes even when the curve is not a straight line.



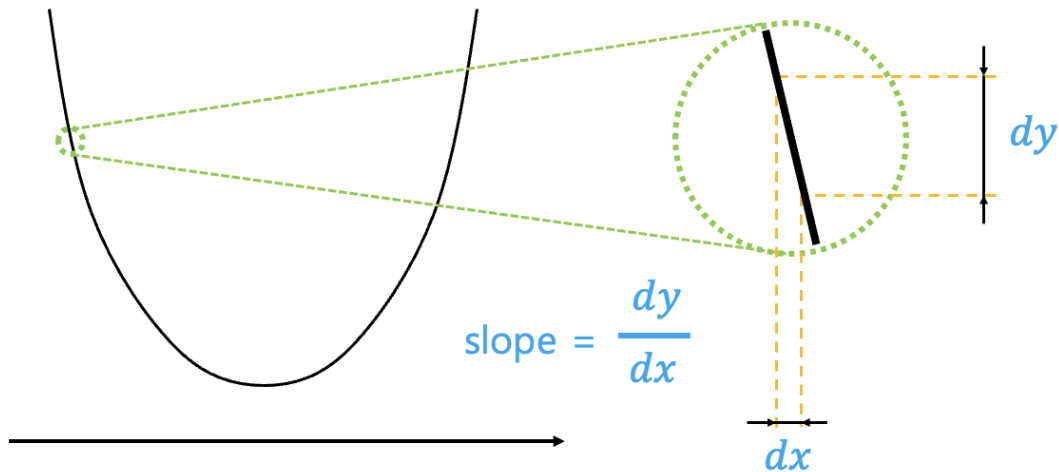
Gradient Descent: Core Idea

With an infinitesimally small change in x , dx ,



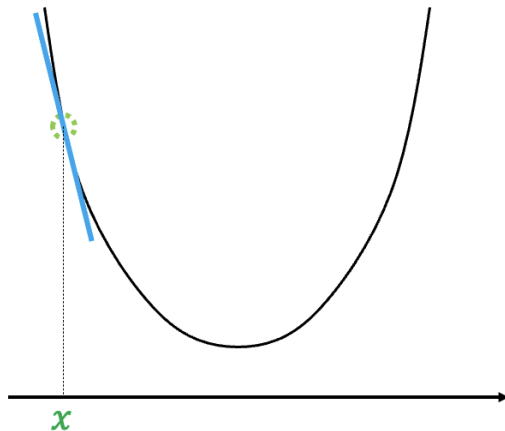
Gradient Descent: Core Idea

and the corresponding infinitesimal change in y , dy , we can calculate the slope.

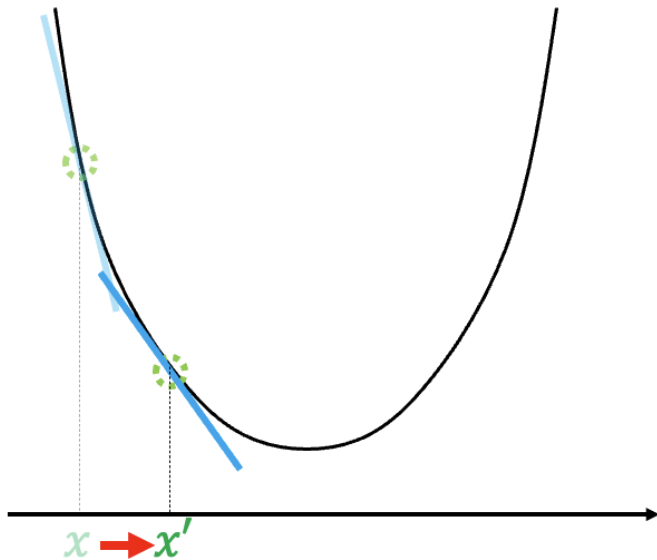


Gradient Descent: Core Idea

The algorithm updates parameters by moving in the **opposite direction of the gradient**. If slope is negative $\rightarrow$ increase the parameter value.

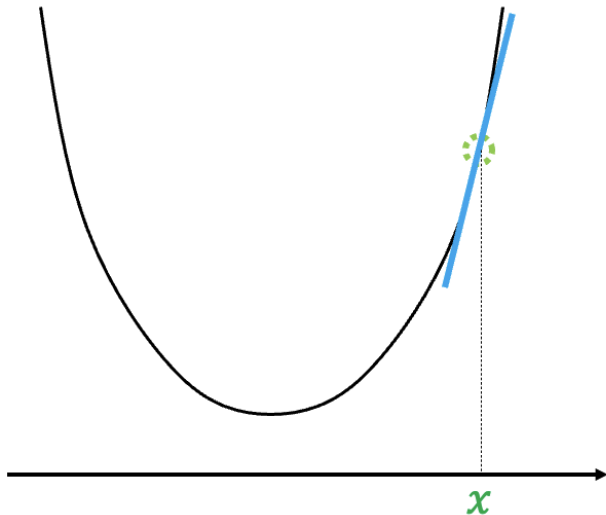


Gradient Descent: Core Idea

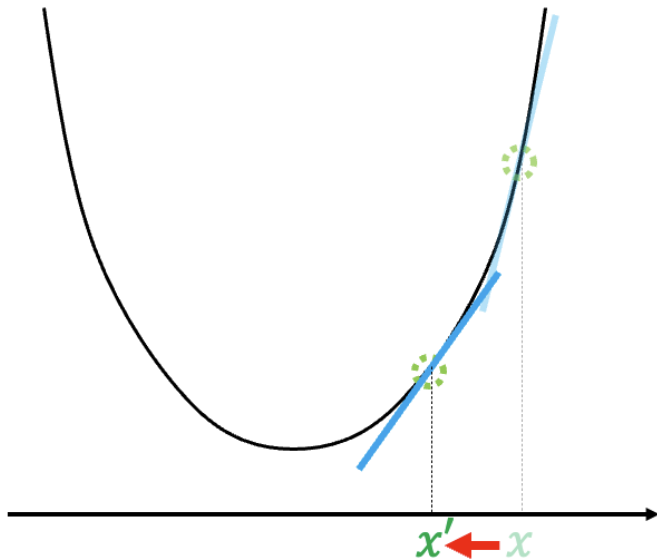


Gradient Descent: Core Idea

If slope is positive $\rightarrow$ decrease the parameter value.

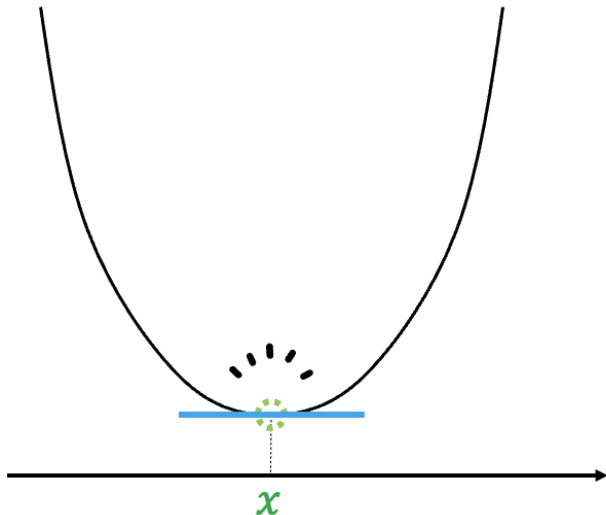


Gradient Descent: Core Idea



Gradient Descent: Core Idea

Iteration continues until the gradient becomes close to zero ($\approx$ minimum loss).



Updating the Weights

Once we know the gradient, we **can update each weight**:

$$w_{\text{new}} = w_{\text{current}} - \eta \frac{\partial L}{\partial w}$$

- $\frac{\partial L}{\partial w}$: how the loss changes with respect to the weight
- η : learning rate – controls the size of each update

Gradient descent uses the gradients to update the weights in the direction that reduces the loss.

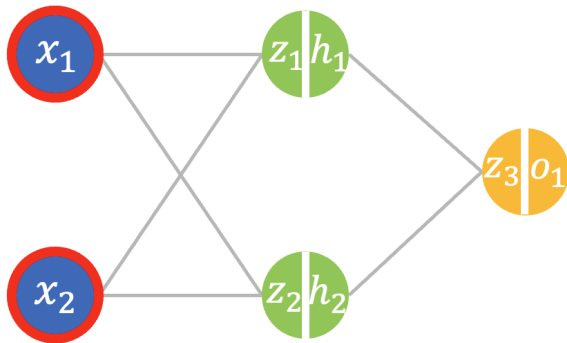
Backpropagation

Now, we have learned everything we need to understand how backpropagation.



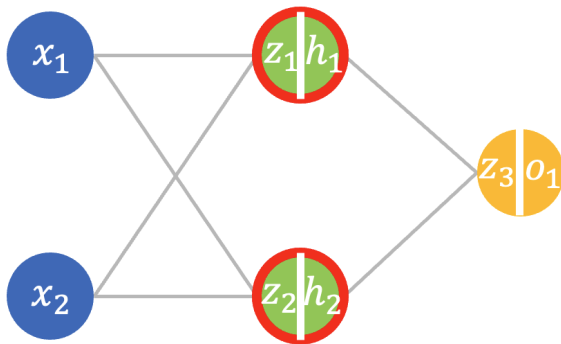
Setup: Input Layer

To understand the core of the backpropagation algorithm, we assume a simple multilayer neural network with two neurons in the input layer.



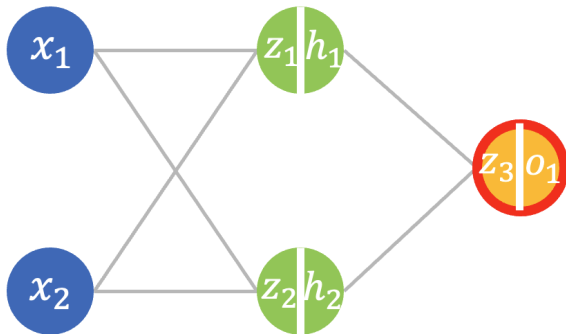
Setup: Hidden Layer

The hidden layer contains two neurons.



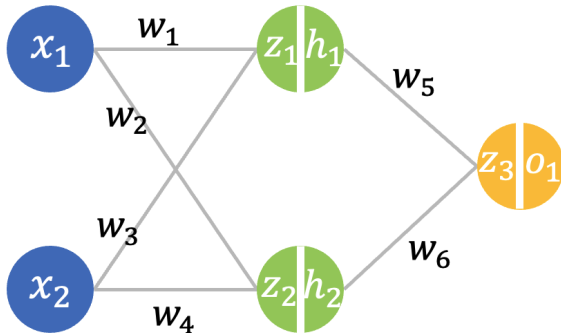
Setup: Output Layer

The output layer has one neuron.



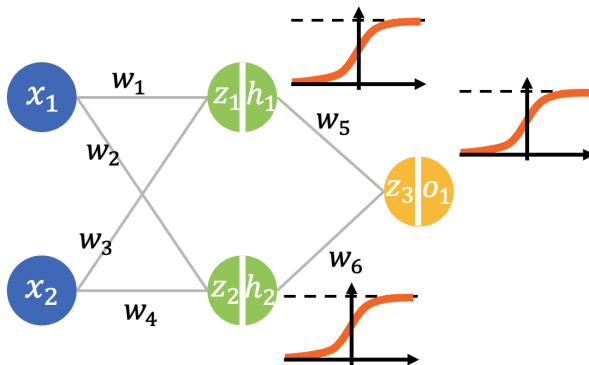
Simple Multilayer Neural Network

There are some weights.



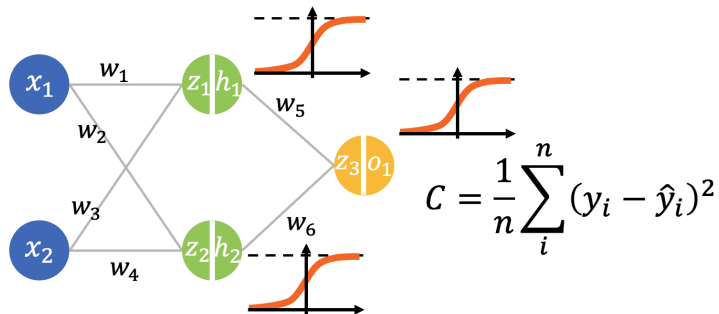
Activation Function

The activation function is the sigmoid.



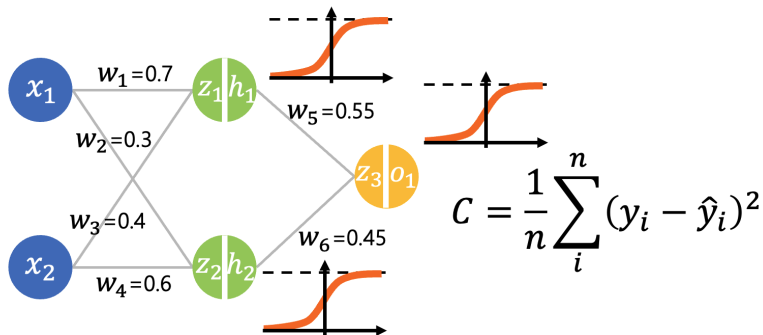
Loss Function

The loss function is Mean Squared Error (MSE).



Weight Initialization and Learning Rate

At the beginning, we suppose that all weights are initialized randomly.
The learning rate is set to 0.1.



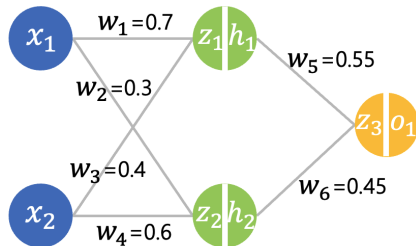
Learning Process

1. **Feedforward:** compute outputs
2. **Loss calculation:** evaluate error
3. **Backpropagation:** propagate errors backward

The process of finding the best parameters is called **learning (optimization)**.

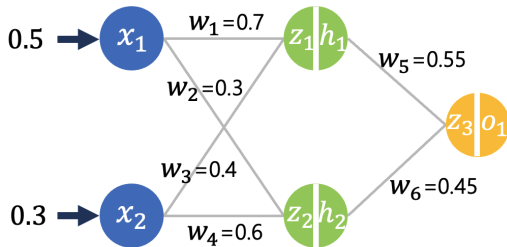
Step 1: Feedforward

The first step is the feedforward stage.



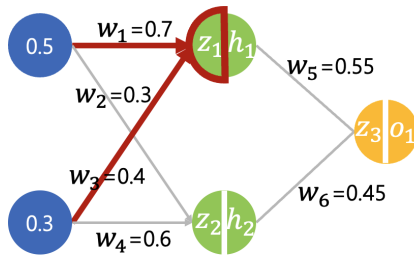
Input Values

The inputs are given as follows.



Weighted Sum to Hidden Node (1)

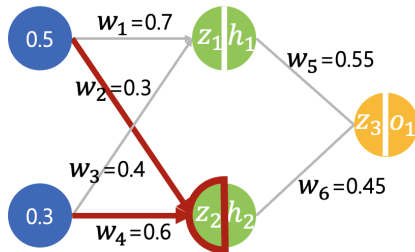
Each input is multiplied by its connection weight, and the results are summed into the hidden layer node.



$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7 + 0.3 \times 0.4 = \mathbf{0.47}$$

Weighted Sum to Hidden Node (2)

Each input is multiplied by its connection weight, and the results are summed into the hidden layer node.

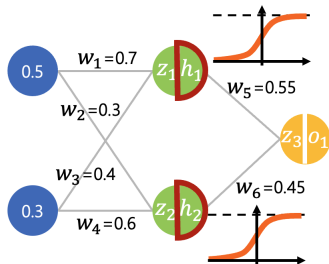


$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7 + 0.3 \times 0.4 = \mathbf{0.47}$$

$$z_2 = x_1 w_2 + x_2 w_4 = 0.5 \times 0.3 + 0.3 \times 0.6 = \mathbf{0.33}$$

Activation at Hidden Node

The activation function is applied to the hidden layer node.



$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7 + 0.3 \times 0.4 = 0.47$$

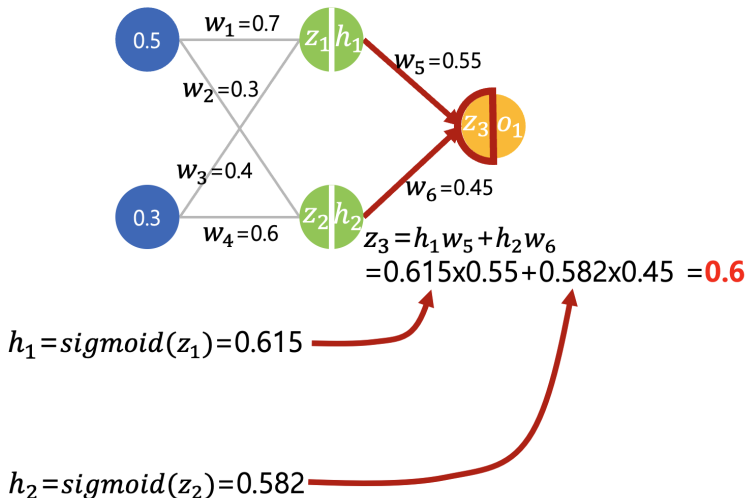
$$h_1 = \text{sigmoid}(z_1) = \mathbf{0.615}$$

$$z_2 = x_1 w_2 + x_2 w_4 = 0.5 \times 0.3 + 0.3 \times 0.6 = 0.33$$

$$h_2 = \text{sigmoid}(z_2) = \mathbf{0.582}$$

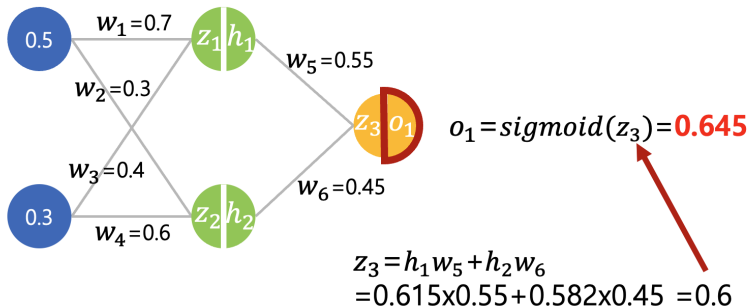
Weighted Sum to Output Neuron

The weighted inputs are summed and passed into the output layer neuron.



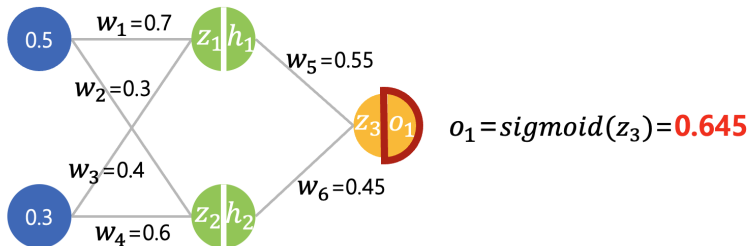
Final Output

The sigmoid function at the output layer produces the final output value.



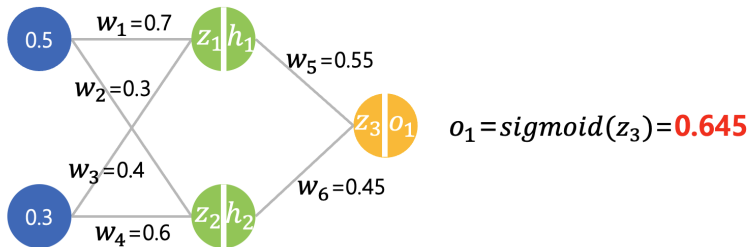
Feedforward Completed

The feedforward stage is now complete.



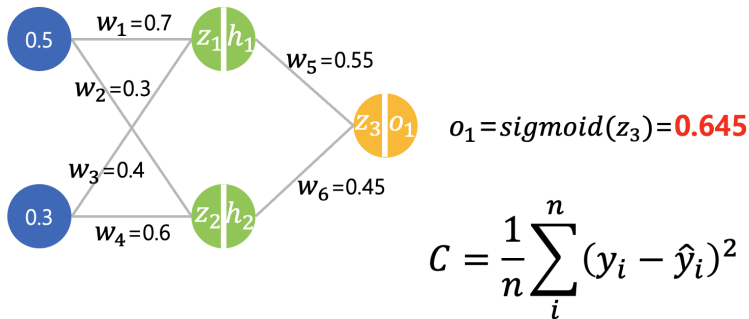
Step 2: Loss Calculation

The second step is the loss calculation stage.



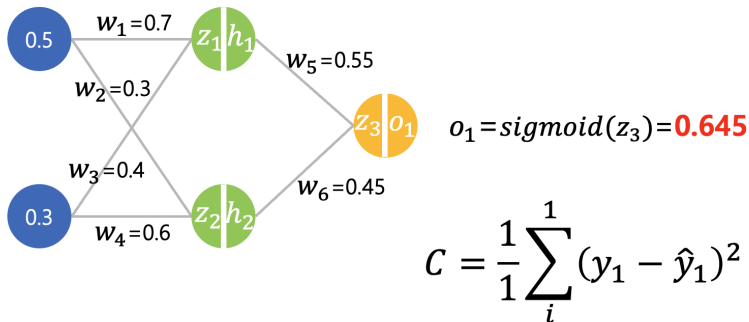
Applying the Loss Function

Since we use MSE as the loss function, the output is substituted into the MSE formula.



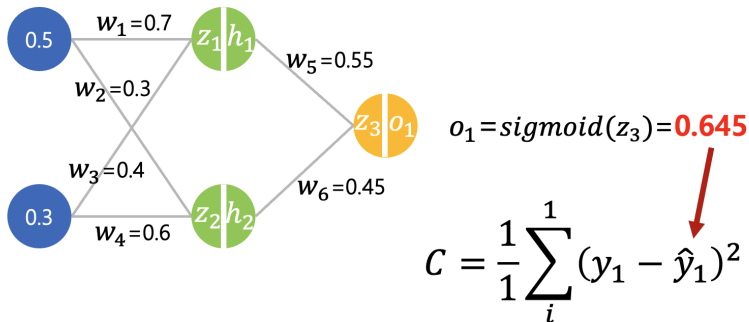
Single Output Neuron

Because there is only one actual output neuron, $n = 1$.



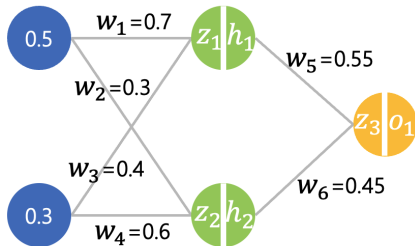
Substituting the Output Value

The predicted value 0.645 is substituted into the MSE.



Assumed Target Value

Suppose the actual target value is 1.

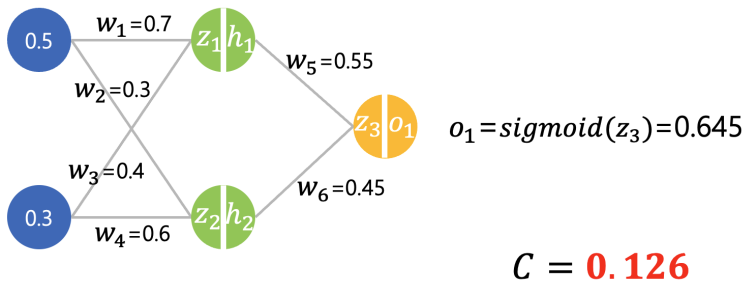


$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

$$C = \frac{1}{1} \sum_i^1 (1 - 0.645)^2$$

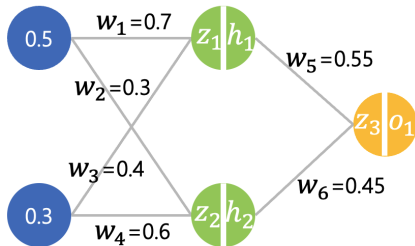
Error Calculation

The error (C) is then calculated.



Step 3: Backpropagation

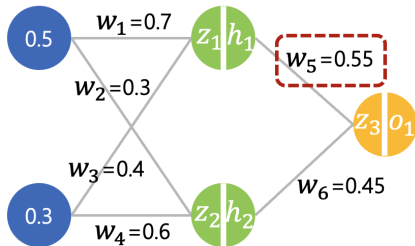
The third step is the backpropagation stage.



$$C = 0.126$$

Updating Weight w_5

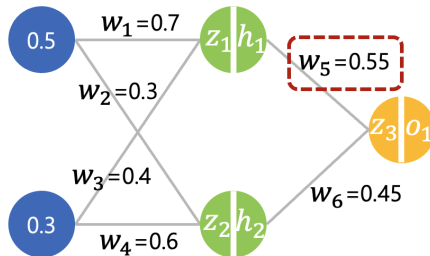
Using backpropagation, we update the weight w_5 .



$$C = 0.126$$

Weight Update Rule

Recall the weight update formula in gradient descent.



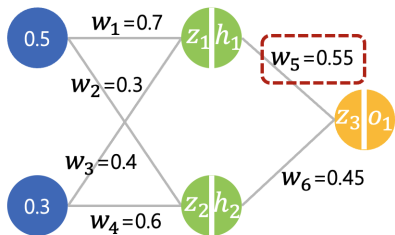
Gradient descent learning rule:

$$w_{\text{new}} = w_{\text{current}} - \eta \cdot \frac{\partial L}{\partial w}$$

- L : loss function
- $\frac{\partial L}{\partial w}$: gradient of the loss function w.r.t. weight

Derivative for w_5

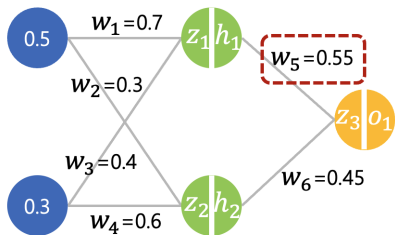
Therefore, to update w_5 , we need to compute the following derivative.



$$\frac{\partial C}{\partial w_5}$$

Using the Chain Rule

Since this derivative cannot be computed directly, we apply the chain rule.



$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Chain Rule

The chain rule is the core of the backpropagation algorithm.

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Unknown Relationship

When we want to compute the derivative of two variables but do not know their direct relationship,

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Expanding with Known Derivatives

we can expand the expression step by step using known partial derivatives, solving the parts to obtain the overall derivative.

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Eliminating Intermediate Variables

In this way, intermediate variables are eliminated, leaving only the relationship we want to compute.

$$\frac{\frac{\partial C}{\partial w_5}}{\frac{\partial C}{\partial w_5}} = \frac{\frac{\partial C}{\partial \phi_1}}{\frac{\partial \phi_1}{\partial z_3}} \cdot \frac{\frac{\partial \phi_1}{\partial z_3}}{\frac{\partial z_3}{\partial w_5}}$$

Chain Rule: An Analogy

How many times faster is the cheetah than the human?

I don't know...

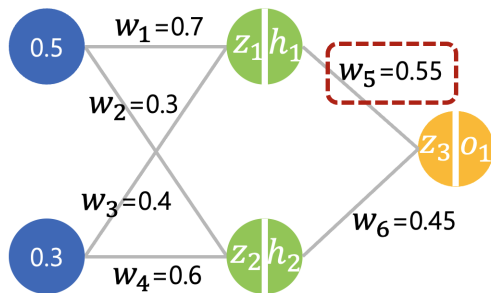
But if you know:

- a cheetah is twice as fast as a lion,
- a lion is twice as fast as a bear,
- and a bear is 1.5 times faster than a *human*

How many times faster is the cheetah than the human?

Breaking Down the Parts

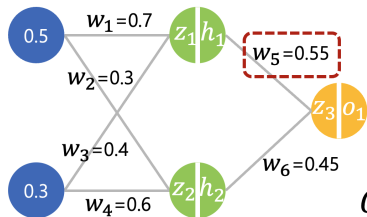
Therefore, we calculate the value by computing each part step by step.



$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

First Derivative

First, we compute the first derivative.

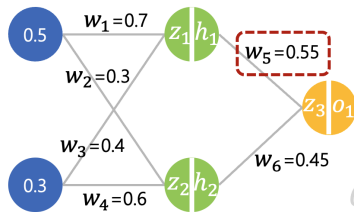


$$C = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Case of $n = 1$

Since $n = 1$,

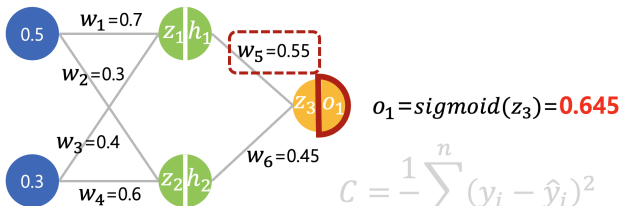


$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

$$C = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$
$$C = (y - o_1)^2$$

Step-by-Step Calculation (1)

Proceeding step by step, we obtain:



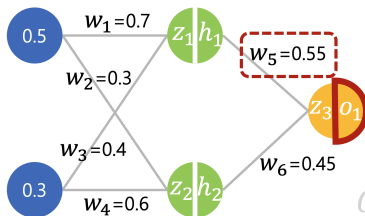
$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

$$C = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

$$C = (y - o_1)^2$$

$$\frac{\partial C}{\partial o_1} = 2(y - o_1)^{2-1} * (-1)$$

Step-by-Step Calculation (2)



$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

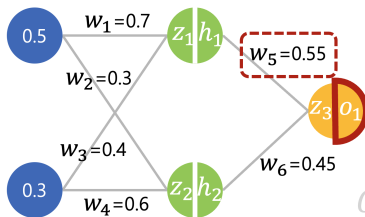
$$C = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

$$C = (y - o_1)^2$$

$$\frac{\partial C}{\partial o_1} = -2(1 - 0.645)$$

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Step-by-Step Calculation (3)



$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

$$C = \frac{1}{n} \sum_i^n (y_i - \hat{y}_i)^2$$

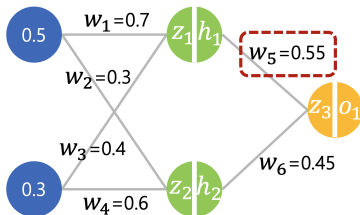
$$C = (y - o_1)^2$$

$$\begin{aligned} \frac{\partial C}{\partial o_1} &= -2(1 - 0.645) \\ &= -0.71 \end{aligned}$$

$$\frac{\partial C}{\partial w_5} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Second Derivative

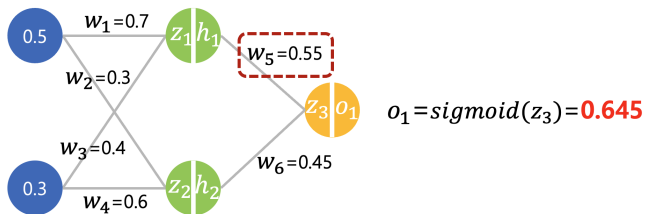
To compute the second derivative,



$$\frac{\partial C}{\partial w_5} = -0.71 \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Sigmoid in Feedforward

Recall that during feedforward, we applied the sigmoid function to z_3 to obtain the output o_1 .



$$\frac{\partial \mathcal{C}}{\partial w_5} = -0.71 \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

Sigmoid Formula

The mathematical formula of the sigmoid function is as follows:

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

Using O and Z Variables

When expressed with O and Z variables, we obtain:

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$O(z) = \frac{1}{1 + e^{-z}}$$

$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

Derivative of Sigmoid

The derivative of the sigmoid function is:

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$O(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial O}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{1 + e^{-z}} \right)$$

$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

Simplified Expression

It can also be expressed as:

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$O(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial O}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{1 + e^{-z}} \right) = O(z)(1 - O(z))$$

$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

Computing the Sigmoid Derivative

Since we already know the value of O_1 , we can compute the derivative of the sigmoid.

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$O(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial O}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{1 + e^{-z}} \right) = O(z)(1 - O(z))$$

$$o_1 = \text{sigmoid}(z_3) = \mathbf{0.645}$$

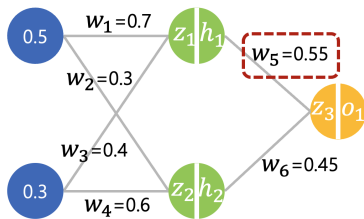

Step-by-Step Expansion (1)

$$S(x) = \frac{1}{1 + e^{-x}}$$

$$O(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial O}{\partial z} = \frac{\partial}{\partial z} \left(\frac{1}{1 + e^{-z}} \right) = 0.645(1 - 0.645) = \mathbf{0.229}$$

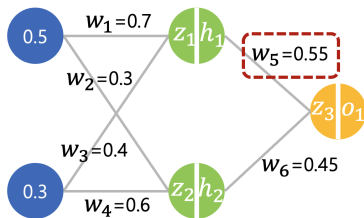
Step-by-Step Expansion (2)



$$\frac{\partial C}{\partial w_5} = -0.71 \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_5}$$

$0.645(1 - 0.645) = \mathbf{0.229}$

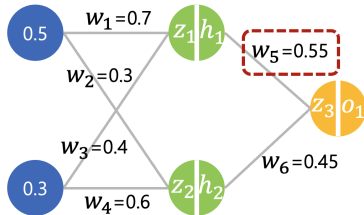
Step-by-Step Expansion (3)



$$\frac{\partial \mathcal{C}}{\partial w_5} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_5}$$

Third Term

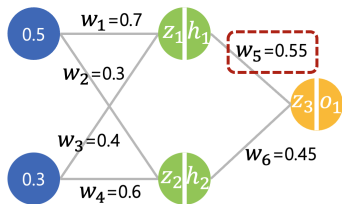
For the third term, we compute:



$$\frac{\partial \mathcal{C}}{\partial w_5} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_5}$$

Formula for z

Using the formula for z,

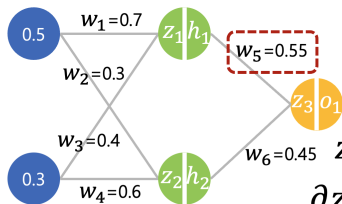


$$z_3 = h_1 w_5 + h_2 w_6$$

$$\frac{\partial C}{\partial w_5} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_5}$$

Partial Derivative of z_3

Taking the partial derivative of z_3 with respect to w_5 directly gives h_1 .



$$z_3 = h_1 w_5 + h_2 w_6$$

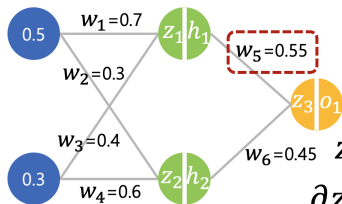
$$\frac{\partial z_3}{\partial w_5} = h_1 \cancel{w_5} + \cancel{h_2 w_6}$$

$$= h_1$$

$$\frac{\partial C}{\partial w_5} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_5}$$

Value of h_1

From the feedforward calculation, $h_1 = 0.615$.



$$\begin{aligned} \frac{\partial C}{\partial w_5} &= -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_5} \\ \frac{\partial z_3}{\partial w_5} &= h_1 \cancel{w_5} + \cancel{h_2 w_6} \\ &= h_1 = 0.615 \end{aligned}$$

Substituting Values

Now, substituting the values, we obtain:

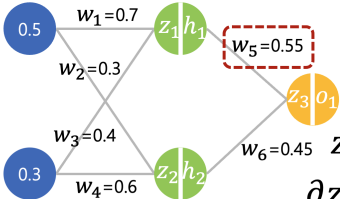


Diagram illustrating a neural network structure with 2 input nodes, 2 hidden nodes, and 1 output node. The weights are labeled on the connections:

- Input 1 (0.5) to Hidden 1 ($z_1 | h_1$): $w_1 = 0.7$
- Input 1 (0.5) to Hidden 2 ($z_2 | h_2$): $w_2 = 0.3$
- Input 2 (0.3) to Hidden 1 ($z_1 | h_1$): $w_3 = 0.4$
- Input 2 (0.3) to Hidden 2 ($z_2 | h_2$): $w_4 = 0.6$
- Hidden 1 ($z_1 | h_1$) to Output ($z_3 | o_1$): $w_5 = 0.55$ (highlighted in a red dashed box)
- Hidden 2 ($z_2 | h_2$) to Output ($z_3 | o_1$): $w_6 = 0.45$

The output calculation is shown as:

$$z_3 = h_1 w_5 + h_2 w_6$$

The partial derivative calculation is shown as:

$$\frac{\partial z_3}{\partial w_5} = h_1 \cancel{w_5} + \cancel{h_2 w_6}$$

The final result is:

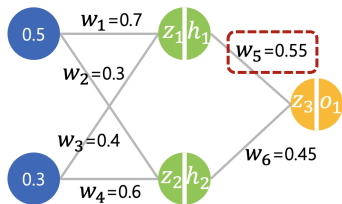
$$= h_1 = 0.615$$

The overall derivative calculation is shown as:

$$\frac{\partial C}{\partial w_5} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_5}$$

Gradient of the Loss Function

Finally, we can compute the gradient of the loss function.



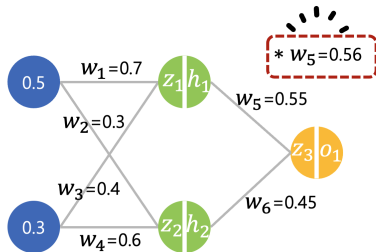
$$\frac{\partial C}{\partial w_5} = -0.71 \cdot 0.229 \cdot 0.615 = -0.1$$

Weight Update with Gradient Descent

According to the gradient descent learning rule:

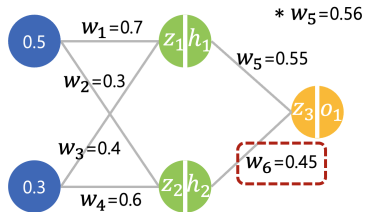
$$w_{\text{new}} = w_{\text{current}} - \eta \cdot \frac{\partial L}{\partial w}$$

The new weight = $0.55 - (-0.1) \cdot 0.1 = 0.56$



Updating Weight w_6

Now, let us update w_6 as well!



$$\frac{\partial \mathcal{C}}{\partial w_6}$$

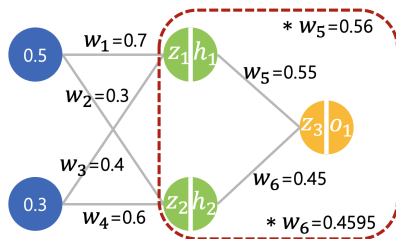
► See Appendix!

Weight Update with Gradient Descent

According to the gradient descent learning rule:

$$w_{\text{new}} = w_{\text{current}} - \eta \cdot \frac{\partial L}{\partial w}$$

The new weight = $0.45 - (-0.095) \cdot 0.1 = 0.4595$



Other Weights

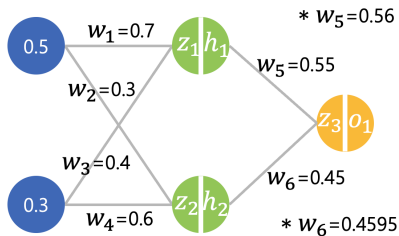
The remaining weights in the first layer are updated in the same way.

* $w_1=0.7010$

* $w_2=0.3009$

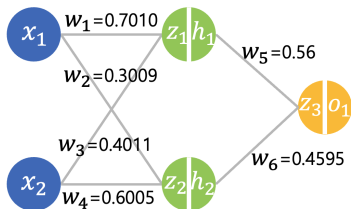
* $w_3=0.4011$

* $w_4=0.6005$



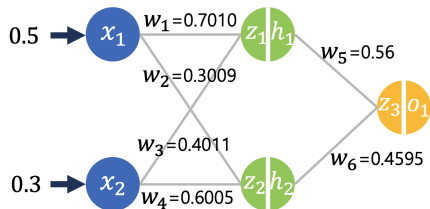
Checking Error After Backpropagation

Now, let us check whether the network error has decreased after backpropagation.



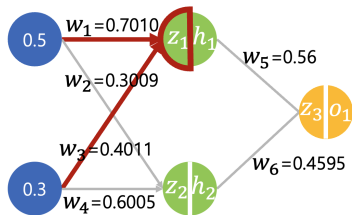
Same Input Values

We feed in the same input values again.



Hidden Node z_1

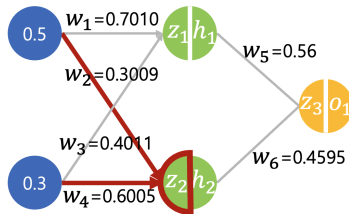
Weighted sums are passed into hidden node z_1 .



$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7010 + 0.3 \times 0.4011 = \mathbf{0.4708}$$

Hidden Node z_2

Weighted sums are passed into hidden node z_2 .

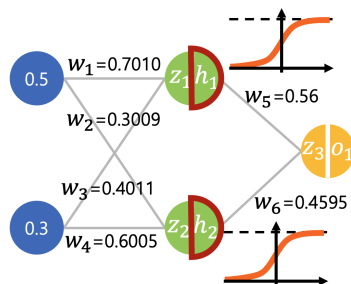


$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7010 + 0.3 \times 0.4011 = \mathbf{0.4708}$$

$$z_2 = x_1 w_2 + x_2 w_4 = 0.5 \times 0.3009 + 0.3 \times 0.6005 = \mathbf{0.3306}$$

Hidden Layer Activation

The activation function is applied at the hidden nodes.



$$z_1 = x_1 w_1 + x_2 w_3 = 0.5 \times 0.7010 + 0.3 \times 0.4011 = \mathbf{0.4708}$$

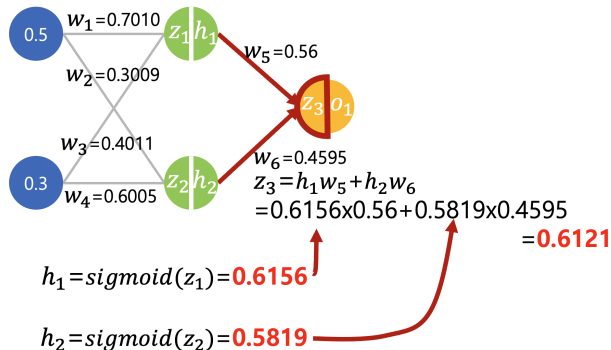
$$h_1 = \text{sigmoid}(z_1) = \mathbf{0.6156}$$

$$z_2 = x_1 w_2 + x_2 w_4 = 0.5 \times 0.3009 + 0.3 \times 0.6005 = \mathbf{0.3306}$$

$$h_2 = \text{sigmoid}(z_2) = \mathbf{0.5819}$$

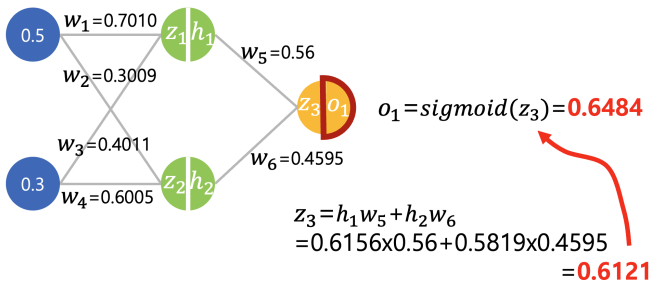
Output Node Weighted Sum

The weighted inputs are summed at the output node.



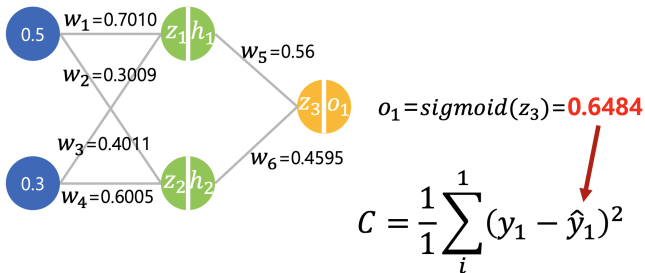
Final Output

Finally, the sigmoid function at the output layer produces the final output.

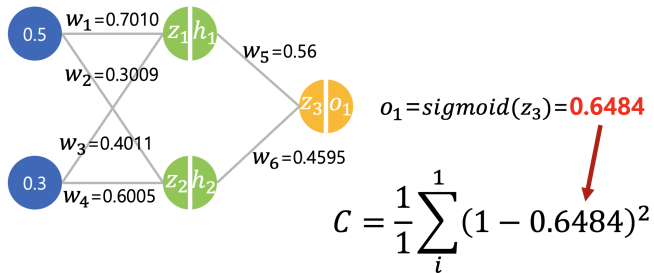


Loss Calculation (1)

The output value is substituted into the loss function.

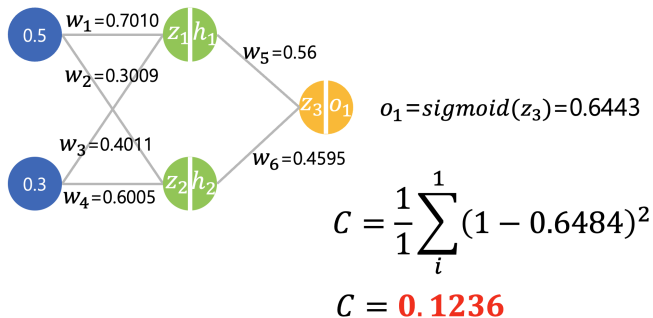


Loss Calculation (2)



Error Comparison

Comparing with the previous error $C = 0.126$, the error is reduced.



Training Process

After this, the same cycle of feedforward, loss calculation, and backpropagation is repeated until the error reaches a minimum, at which point training stops.

From Scalar Derivatives to Matrices

So far, we have considered the case of a **single output**, where the gradient with respect to the weights is just a scalar derivative. But when the function has **multiple outputs**, the derivative generalizes to a **matrix**.

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

Example: Neural Networks for Dependency Parsing

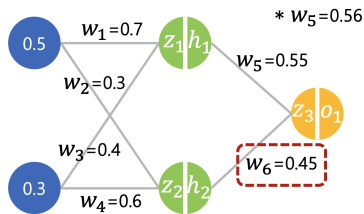
- Dependency parsing identifies grammatical relationships between words in a sentence (e.g., subject, object, modifier).
- Chen and Manning (2014) used a feed-forward neural network to predict parsing actions.



Appendix

Using the Chain Rule Again

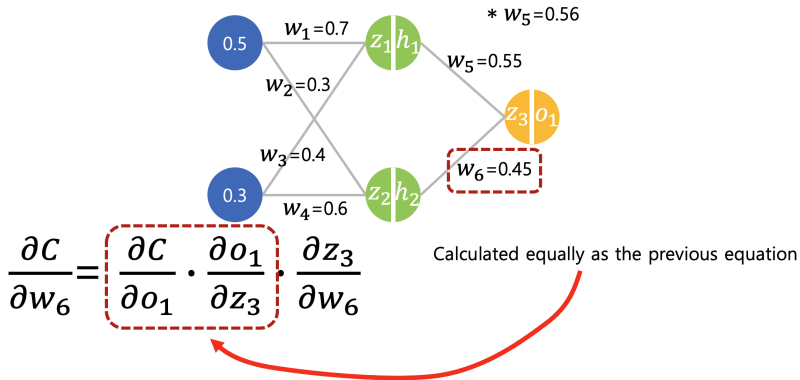
By the chain rule, we can similarly derive the expression.



$$\frac{\partial C}{\partial w_6} = \frac{\partial C}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_6}$$

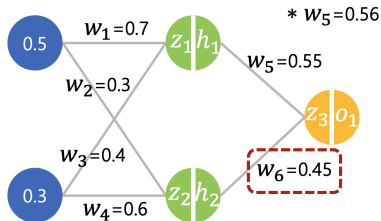
Same as w_5 Formula

The previous steps are the same as for the w_5 calculation.



Substituting Values

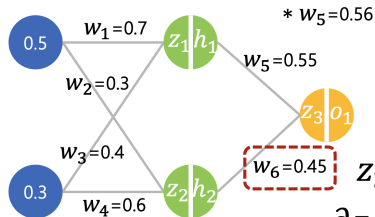
Now, substituting the values, we obtain:



$$\frac{\partial C}{\partial w_6} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_6}$$

Third Term

The third term once again becomes h_2 .

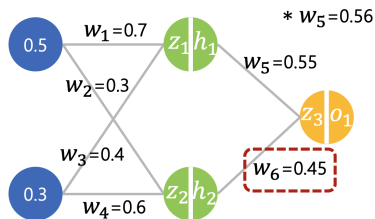


$$\frac{\partial C}{\partial w_6} = -0.71 \cdot 0.229 \cdot \frac{\partial z_3}{\partial w_6}$$

$$\begin{aligned} z_3 &= h_1 w_5 + h_2 w_6 \\ \frac{\partial z_3}{\partial w_6} &= \cancel{h_1 w_5} + \cancel{h_2 w_6} \\ &= h_2 = \mathbf{0.582} \end{aligned}$$

Gradient for w_6

The gradient of the loss function is -0.095 .



$$\frac{\partial C}{\partial w_6} = -0.71 \cdot 0.229 \cdot 0.582 = -0.095$$